

Friends

Det finns vissa fall då man skulle vilja att klasser skulle kunna bryta mot de skyddsregler som gäller för privata data i klasser. Med detta avses att en viss klass eller funktion skulle kunna accessera de medlemmar och metoder som är deklarerade som `private` i en klass. En situation då ett dylikt förfarande kunde vara praktiskt är då vi t.ex. har en klass `Image` som innehåller en bild, och en klass `ImageViewer` som visar bilder på skärmen. I normala fall kan inga externa exntiter ändra på bildens innehåll, d.v.s. pixeldata, men vi skulle vilja att bildvisaren har vissa möjligheter att korrigera t.ex. storleken eller små fel som uppstår vid förstoring eller rotation av bilden. Det är inte möjligt utan specialarrangemang. Vi kunde tänka oss följande förenklade definitioner på klasserna.

```
class Image {
public:
    // konstruktör
    Image (unsigned int Width, unsigned int Height, int * Pixels);

    // visa bilden
    void show ();

private:
    // bildens storlek
    unsigned int m_Width;
    unsigned int m_Height;

    // pixeldata (i något internt format)
    int * m_Pixels;
};

class ImageViewer {
public:
    // konstruktör
    ImageViewer ();

    // ladda en bild från disk
    Image * load (string Filename);

    // visa en bild
    void show (const Image & Picture);

    // korrigera en bild enligt någon algorithm
    void fix (Image & Picture);

private:
    // lista av alla bilder
    list m_Images;
};

Image * ImageViewer::load (string Filename) {
    // läs data från filen in i lokala variabler
    unsigned int Width = ...;
    unsigned int Height = ...;
    int * Data = ...;

    // skapa ny bild
    Image * Picture = new Image ( Width, Height, Data );
}
```

```

// fixa små fel i bilden
fix ( *Picture );

// returera den nya bilden vi laddat in
return New;
}

void ImageViewer::fix (Image & Picture) {
    // accessera pixeldata för 'Picture' enligt fin algoritm
    ...
}

```

Vi går inte närmare in på själva koden för dessa klasser, utan koncentrerar oss på gränssnittet de presenterar utåt. Klassen `Image` har en privat medlem `m_Pixels` som innehåller alla pixlar för en bild. Metoden `fix()` i `ImageViewer` vill kunna direkt accessera pixeldata i bilden `Picture`. Det är inte möjligt, utan kompilatorn kommer att ge oss ett felmeddelande om att vi inte har rätt att accessera privata data i `Image`. Lösningen är att `Image` deklarerar klassen `ImageViewer` som en pålitlig "vän" (friend). En vänklass har rätt att accessera private data i en annan.

Bryter inte detta mot grundprinciperna i objektorientering då? Klassers privata data skall vara privat, anser purister. Inte egentligen eftersom det i detta fall är `Image` som måste explicit säga att `ImageViewer` är dess friend. Det går inte andra vägen. Den klass som blir accesserad måste explicit tillåta andra klasser att göra detta. Så det är frågan om ett medvetet designbeslut man gjort då man skrivit en klass. För att deklarera en klass för en friend används följande allmänna form:

```
friend class klassnamn;
```

som placeras någonstans *inne i* den klass vars privata data skall kunna accesseras av `klassnamn`. I vårt fall kunde vi ändra definitionen av klassen `Image` till:

```

class Image {
    // deklarerar en vän
    friend class ImageViewer;

public:
    // konstruktor
    Image (unsigned int Width, unsigned int Height, int * Pixels);

    // visa bilden
    void show ();

private:
    // bildens storlek
    unsigned int m_Width;
    unsigned int m_Height;

    // pixeldata (i något internt format)
    int * m_Pixels;
};

```

Enda skillnaden är friend-deklarationen. Nu kan `ImageViewer` accessera alla privata data i `Image` som om de vore public.

Friend-metoder

Om vi tittar lite närmare på vår klass `ImageViewer` märker vi snabbt att det är endast metoden `fix()` som behöver direkt access till `Image`. Vi kunde alltså med fördel begränsa så att endast denna metod

kan accessera privata data, istället för att alla metoder kan göra det:

```
class Image {  
    // deklarerar en vän  
    friend class ImageViewer::fix (Image & Picture);  
  
public:  
    ...  
};
```

Vi skriver nu in hela metodens namn med klassnamn, metodnamn och alla parametrar, exakt som den deklarerar i `ImageViewer`. På detta sätt kan t.ex. `load()` inte längre accessera privata data. Det kan vara en god ide att begränsa denna access till exakt de metoder som behöver den, för att inte i misstag i andra metoder accessa privata data när det egentligen inte behövs. Att använda `friend` är inte en lösning på problemet med en dåligt konstruerad klasshierarki. I så fall är det bättre att designa om sina klasser.

En klass kan ha multipla vänner om så skulle behövas. Man placerar bara en ny definition någonstans i klassdefinitionen. Det är ingen skillnad var man placerar `friend`-deklarationer, d.v.s. inom `public`, `protected` eller `private`, så länge som de är separata deklarationer.

[Förutgående](#)

Mera om klasser

[Hem](#)

[Upp](#)

[Nästa](#)

Polymorfism